Errand Agent: design note

Core rule: the model reads and proposes. Only deterministic code changes a booking, after an approval the model cannot produce. FastAPI, PostgreSQL, Redis, Google ADK and Gemini, run by one Compose command.

State model. One `workflows` row per errand: `INTAKE → SEARCHING → COMPARING → AWAITING_APPROVAL → EXECUTING → VERIFYING → DONE`, plus `DISRUPTED`, `FAILED` and `LIMIT_REACHED`. A transition table rejects illegal moves, and a row lock with a version check stops overwrites. The model has no tool that sets state. Nothing that executed reaches `DONE` without a status check. Every run ends in one of six outcomes, with a reply, inside hard limits: 8 model calls, 12 tool calls, 120 s.

Approval enforcement. The model has no `book`, `cancel` or `reschedule` tool. `propose_booking` takes a hold and writes an approval row with a SHA-256 hash of the action, user, provider, slot, prices, hold and expiry. The card comes from that row, never from model text or reviews. A yes is a click, or a whole-message match read from the raw message before the model runs, so "yes, but Saturday" is a change. Every tool goes through one gateway, which accepts side effects only from the saga, with a token rebuilt from current data. A moved price fails the hash: no call is sent, and the user gets a fresh card. Security rests on what the model can do, so an injected review has nothing to abuse.

Idempotency and recovery. Consuming the approval, writing the attempt and queuing the job share one transaction. The key is `book:<approval_id>`, and the attempt commits before each provider call. After any result, `get_booking_status` decides: booked means no retry, not booked retries with the same key, at most 3 calls in all. PostgreSQL is the truth and Redis only carries work. So a sweep recovers a killed worker through leases, and a Redis wipe through the outbox. Provider cancels use the same path: a poll detects them, code decides whether to notify, nothing is held, and rebooking needs a new card.

Memory. Facts live in `memories`, separate from any chat. A fact is explicit only if the quote is the user's own and states the value. Otherwise it stays out of context until the user confirms. Correcting supersedes; forgetting scrubs the chat, errand, logs and traces, then deletes the ADK session. ADK's history is replaced on every call by `ContextAssembler`: filter in SQL, rank by service, trim to 8,000 estimated tokens, and log to `context_log` first. If the log fails, the call does not run.

Observability. One run ID follows a booking from `api` to `worker`, in `trace_events` and one JSONL file per run: state moves, model calls, sanitized tool calls, latency, tokens and cost. `make evals` runs 38 scenarios offline on real PostgreSQL, Redis and worker processes, and reads invariants I1 to I8 from the databases after each one.

Trade-offs. One agent, because guardrails live in code and more agents add cost, not safety. ADK runs only the inner loop, so it stays replaceable. Ranking is deterministic, with no embeddings, so tests repeat. **Limits:** scripted evals test our rules, not Gemini's judgement, and the false-claim guard reads wording, so success text comes only from a verified template.

Trace: timeout after success (raw: `docs/trace-timeout-after-success.jsonl`, from eval E11, scripted model, fake clock):

```
 1 api     approval_answer        yes, source: click          <- the user, not the model
 3 api     approval_consumed      key book:9d08...            <- one transaction with the job
 4 api     outcome                verifying                   <- reply: "not confirmed yet"
 5 worker  saga_claimed           book_calls_so_far: 0        <- same run ID, another process
 7 worker  tool_call book         timeout, ambiguous, 3010 ms <- booked, but the reply was lost
 8 worker  state_transition       executing -> verifying      <- no retry on a timeout
 9 worker  get_booking_status     ok                          <- the source of truth
10 worker  booking_status_checked booked, after: timeout      <- so one booking, one call
12 worker  booking_verified       bkg-af582af5cd5e            <- only now may success be said
```